

Logistic Regression – Cell-by-Cell ML Developer Explanation

Based directly on the uploaded Jupyter notebook: “Logistics Regression.ipynb”

Purpose: understand what every cell does, why it is used, what the stored output means, and how the complete classification pipeline works.

1. Project Overview

Business objective: build a classification model that predicts the species of an Iris flower from sepal length, sepal width, petal length, and petal width. The notebook uses Logistic Regression with scikit-learn.

ML workflow: Data Gathering → Data Quality Checks → X/Y Separation → Train/Test Split → Preprocessing → Model Training → Evaluation → Class Distribution → Model Saving → Out-of-Sample Prediction → CSV Export.

2. Key Results from the Notebook

Dataset after duplicate removal	149 rows × 5 columns
Features	4 numeric flower measurements
Target	species (setosa, versicolor, virginica)
Train/Test split	119 training rows / 30 testing rows
Training accuracy	98.32%
Testing accuracy	93.33%
Macro F1-score	92.80%
Target counts	setosa 50, versicolor 50, virginica 49

The notebook also stores a DataConversionWarning during model fitting because ytrain is a one-column DataFrame. A cleaner implementation would use ytrain.squeeze() or ytrain.iloc[:, 0] so the target is a 1D vector.

Cell 0 — MARKDOWN

Explanation

This opening cell defines the notebook topic and business objective. The project is a classification problem: predict the Iris flower species from four measurements.

Cell 1 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 2 — CODE

```
import pandas as pd
path = r"https://raw.githubusercontent.com/sindhura-nk/Datasets/refs/heads/main/iris.csv"
df = pd.read_csv(path)
df.head()
```

Explanation

Loads the Iris CSV into a pandas DataFrame. This is the data-gathering stage; the model needs a structured table before quality checks and feature/target separation.

Stored notebook output

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Cell 3 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 4 — CODE

```
df.shape
```

Explanation

Checks the number of rows and columns. It confirms the dataset size before modeling.

Stored notebook output

```
(150, 5)
```

Cell 5 — CODE

```
df.info()
```

Explanation

Checks column names, non-null counts, data types, and approximate memory usage. This helps identify missing values and whether features are numeric/categorical.

Stored notebook output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
2   petal_length    150 non-null   float64
3   petal_width     150 non-null   float64
4   species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

Cell 6 — CODE

```
df.duplicated().sum()
```

Explanation

Checks for duplicate rows and then removes duplicates. Duplicate records can bias training if the same observation is repeated.

Stored notebook output

```
np.int64(1)
```

Cell 7 — CODE

```
df = df.drop_duplicates()  
df.duplicated().sum()
```

Explanation

Checks for duplicate rows and then removes duplicates. Duplicate records can bias training if the same observation is repeated.

Stored notebook output

```
np.int64(0)
```

Cell 8 — CODE

```
df.isna().sum()
```

Explanation

Checks missing values in every column. The stored output shows zero missing values in the training dataset after duplicate removal.

Stored notebook output

```
sepal_length    0  
sepal_width     0  
petal_length    0  
petal_width     0  
species         0  
dtype: int64
```

Cell 9 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 10 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 11 — CODE

```
X = df.drop(columns='species')
Y = df[['species']]
print("X Records")
display(X.head())
print("Y Records")
display(Y.head())
```

Explanation

Separates predictors X from the target Y. X contains the four flower measurements; Y contains species, the value the model must predict.

Stored notebook output

```
X Records

   sepal_length  sepal_width  petal_length  petal_width
0           5.1           3.5           1.4           0.2
1           4.9           3.0           1.4           0.2
2           4.7           3.2           1.3           0.2
3           4.6           3.1           1.5           0.2
4           5.0           3.6           1.4           0.2
Y Records

   species
0  setosa
1  setosa
2  setosa
3  setosa
4  setosa
```

Cell 12 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 13 — CODE

```
from sklearn.model_selection import train_test_split

xtrain,xtest,ytrain,ytest = train_test_split(X,Y, train_size=0.8, random_state=21)
print("X Training and Testing :", xtrain.shape,xtest.shape)
print("Y Training and Testing :", ytrain.shape,ytest.shape)
```

Explanation

Splits the cleaned data into training and testing sets. The notebook uses 80% for training and 20% for testing with a fixed random_state for reproducibility.

Stored notebook output

```
X Training and Testing : (119, 4) (30, 4)
Y Training and Testing : (119, 1) (30, 1)
```

Cell 14 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 15 — CODE

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer

num_pipe = make_pipeline(SimpleImputer(strategy='median')).set_output(transform="pandas")
num_pipe
```

Explanation

Creates a preprocessing pipeline with median imputation. The pipeline learns the median from training data and can then transform future data consistently.

Stored notebook output

```
Pipeline(steps=[('simpleimputer', SimpleImputer(strategy='median'))])
```

Cell 16 — CODE

```
# fit it on training data
num_pipe.fit(xtrain)
```

Explanation

Fits the preprocessing pipeline on training data only. This is important because preprocessing statistics should not be learned from the test set.

Stored notebook output

```
Pipeline(steps=[('simpleimputer', SimpleImputer(strategy='median'))])
```

Cell 17 — CODE

```
# transform it on testing as well as training data
xtrain_pre = num_pipe.transform(xtrain)
xtest_pre = num_pipe.transform(xtest)

xtrain_pre.head()
```

Explanation

Applies the fitted cleaning pipeline to training and testing data. The result remains in pandas format because `set_output(transform='pandas')` was used.

Stored notebook output

	sepal_length	sepal_width	petal_length	petal_width	
41	4.5	2.3	1.3	0.3	
130	7.4	2.8	6.1	1.9	
70	5.9	3.2	4.8	1.8	
46	5.1	3.8	1.6	0.2	
125	7.2	3.2	6.0	1.8	

Cell 18 — CODE

```
xtest_pre.head()
```

Explanation

This code cell performs the next operation in the notebook's ML workflow. Its exact purpose is determined by the variables and functions shown in the source code.

Stored notebook output

	sepal_length	sepal_width	petal_length	petal_width	
92	5.8	2.6	4.0	1.2	
44	5.1	3.8	1.9	0.4	
7	5.0	3.4	1.5	0.2	
21	5.1	3.7	1.5	0.4	
95	5.7	3.0	4.2	1.2	

Cell 19 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 20 — CODE

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(xtrain_pre,ytrain)
```

Explanation

Creates and trains the Logistic Regression classifier. Logistic Regression is a classification algorithm that estimates class probabilities and converts them into class predictions.

Stored notebook output

```
e:\DATA SCIENTIST\ML\repository\venv\lib\site-packages\sklearn\utils\validation.py:1406: DataConversionWarnin
y = column_or_1d(y, warn=True)

LogisticRegression()
```

Cell 21 — CODE

```
# training scores(accuracy)
model.score(xtrain_pre,ytrain)
```

Explanation

Calculates the classifier's accuracy on the supplied data. The notebook checks both training and test accuracy.

Stored notebook output

```
0.9831932773109243
```

Interpretation: 0.98319 means about 98.32% of training observations were classified correctly.

Cell 22 — CODE

```
# testing scores(accuracy)
model.score(xtest_pre,ytest)
```

Explanation

Calculates the classifier's accuracy on the supplied data. The notebook checks both training and test accuracy.

Stored notebook output

```
0.9333333333333333
```

Interpretation: 0.93333 means about 93.33% of unseen test observations were classified correctly. The gap from training accuracy is small, so there is no obvious severe overfitting from these two scores alone.

Cell 23 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 24 — CODE

```
ytest_preds = model.predict(xtest_pre)
ytest_preds[:5]
```

Explanation

Generates predicted class labels for the test observations.

Stored notebook output

```
array(['versicolor', 'setosa', 'setosa', 'setosa', 'versicolor'],
      dtype=object)
```

Cell 25 — CODE

```
ytest.head()
```

Explanation

This code cell performs the next operation in the notebook's ML workflow. Its exact purpose is determined by the variables and functions shown in the source code.

Stored notebook output

```
species
92  versicolor
44      setosa
7      setosa
21      setosa
95  versicolor
```

Cell 26 — CODE

```
from sklearn.metrics import f1_score

f1_score_data = f1_score(ytest,ytest_preds,average='macro')
f1_score_data
```

Explanation

Calculates macro F1-score, giving equal importance to each class. F1 balances precision and recall.

Stored notebook output

```
0.9280303030303031
```

Cell 27 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 28 — CODE

```
from sklearn.metrics import ConfusionMatrixDisplay,classification_report
ConfusionMatrixDisplay.from_estimator(model, xtest_pre,ytest)
```

Explanation

Builds a confusion matrix from the trained estimator. It shows which classes were predicted correctly and where the model confused one species with another.

Stored notebook output

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x12224998730>
<Figure size 640x480 with 2 Axes>
```

Cell 29 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 30 — CODE

```
print(classification_report(ytest,ytest_preds))
```

Explanation

Produces precision, recall, F1-score, and support for each class, plus macro and weighted averages.

Stored notebook output

precision	recall	f1-score	support	
setosa	1.00	1.00	1.00	11
versicolor	1.00	0.83	0.91	12
virginica	0.78	1.00	0.88	7
accuracy			0.93	30
macro avg	0.93	0.94	0.93	30
weighted avg	0.95	0.93	0.93	30

Interpretation: Setosa is classified perfectly in this test split. Versicolor has recall 0.83, meaning some actual versicolor flowers were predicted as another class. Virginica has precision 0.78, meaning some predictions labeled virginica were actually another species.

Cell 31 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 32 — CODE

```
df['species'].value_counts()
```

Explanation

Counts observations in each target class. This is a class-distribution check used to understand whether the target is imbalanced.

Stored notebook output

```
species
setosa      50
versicolor  50
virginica   49
Name: count, dtype: int64
```

Interpretation: the target is essentially balanced, so the notebook does not need an imbalance technique such as SMOTE for this dataset.

Cell 33 — CODE

```
df['species'].value_counts().plot(kind='bar')
```

Explanation

Counts observations in each target class. This is a class-distribution check used to understand whether the target is imbalanced.

Stored notebook output

```
<Axes: xlabel='species'>
```

<Figure size 640x480 with 1 Axes>

Cell 34 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 35 — CODE

```
import joblib
joblib.dump(model, "IrisPredictionModel.joblib")
joblib.dump(num_pipe, "pipeline_cleaning.joblib")
```

Explanation

Serializes the trained model and preprocessing pipeline to files so they can be loaded later without retraining.

Stored notebook output

```
['pipeline_cleaning.joblib']
```

Cell 36 — MARKDOWN

Explanation

This is a notebook section heading. It tells us the stage of the ML workflow that follows.

Cell 37 — CODE

```
xnew = pd.read_csv(r"https://raw.githubusercontent.com/sindhura-nk/Datasets/refs/heads/main/iris_sample.csv")
xnew.head()
```

Explanation

Loads the Iris CSV into a pandas DataFrame. This is the data-gathering stage; the model needs a structured table before quality checks and feature/target separation.

Stored notebook output

	sepal_length	sepal_width	petal_length	petal_width
0	5.5	2.5	4.0	1.3
1	6.9	3.1	5.1	2.3
2	5.1	2.5	3.0	1.1
3	4.4	2.9	NaN	0.2
4	5.9	3.0	5.1	1.8

Cell 38 — CODE

```
pipeline_loaded = joblib.load("pipeline_cleaning.joblib")
xnew_pre = pipeline_loaded.transform(xnew)
xnew_pre.head()
```

Explanation

Loads the saved preprocessing pipeline or model back into memory for inference.

Stored notebook output

	sepal_length	sepal_width	petal_length	petal_width
0	5.5	2.5	4.0	1.3
1	6.9	3.1	5.1	2.3
2	5.1	2.5	3.0	1.1
3	4.4	2.9	4.5	0.2
4	5.9	3.0	5.1	1.8

Cell 39 — CODE

```
mdoel_loaded = joblib.load("IrisPredictionModel.joblib")
Species_Prediction = mdoel_loaded.predict(xnew_pre)
Species_Prediction[:5]
```

Explanation

Loads the saved preprocessing pipeline or model back into memory for inference.

Stored notebook output

```
array(['versicolor', 'virginica', 'versicolor', 'versicolor', 'virginica'],
      dtype=object)
```

Interpretation: the saved model predicts the first five new observations as versicolor, virginica, versicolor, versicolor, and virginica.

Cell 40 — CODE

```
xnew.head()
```

Explanation

This code cell performs the next operation in the notebook's ML workflow. Its exact purpose is determined by the variables and functions shown in the source code.

Stored notebook output

	sepal_length	sepal_width	petal_length	petal_width
0	5.5	2.5	4.0	1.3
1	6.9	3.1	5.1	2.3
2	5.1	2.5	3.0	1.1
3	4.4	2.9	NaN	0.2
4	5.9	3.0	5.1	1.8

Cell 41 — CODE

```
xnew["Predicted_Species"] = Species_Prediction
xnew.head()
```

Explanation

Uses the loaded Logistic Regression model to predict the species for the new samples.

Stored notebook output

	sepal_length	sepal_width	petal_length	petal_width	Predicted_Species
0	5.5	2.5	4.0	1.3	versicolor
1	6.9	3.1	5.1	2.3	virginica
2	5.1	2.5	3.0	1.1	versicolor
3	4.4	2.9	NaN	0.2	versicolor
4	5.9	3.0	5.1	1.8	virginica

Cell 42 — CODE

```
xnew.to_csv("IrisSpeciesPredictedResults.csv", index=False)
```

Explanation

Exports the prediction results to a CSV file for later use.

Cell 43 — CODE

Explanation

This code cell performs the next operation in the notebook's ML workflow. Its exact purpose is determined by the variables and functions shown in the source code.

3. ML Developer Review

What is done well

- The notebook follows a recognizable end-to-end ML pipeline.
- It checks shape, data types, duplicates, and missing values before modeling.
- It separates X and Y before splitting the data.
- It fits preprocessing on training data and reuses it for test/new data.
- It evaluates with accuracy, macro F1, confusion matrix, and classification report.
- It saves both the model and preprocessing pipeline for later inference.
- It demonstrates out-of-sample prediction and exports predictions to CSV.

Important improvements

1. Rename the notebook consistently to "Logistic Regression" (the uploaded filename uses "Logistics Regression").
2. Convert the target to a 1D Series before model fitting, e.g. `ytrain = ytrain.squeeze()`, to avoid the `DataConversionWarning`.
3. The current preprocessing pipeline only performs median imputation. That is valid, but the Iris feature columns are already complete in the training data, so the imputer is mainly useful for demonstrating a reusable missing-value strategy.
4. For a production pipeline, combine preprocessing and Logistic Regression into one Pipeline so preprocessing and prediction are saved as a single object.
5. Add a stratified split when class proportions need to be preserved explicitly. Here the class distribution is already nearly balanced.
6. Add cross-validation for a more robust estimate instead of relying on one train/test split.
7. Use clearer variable names such as `X_train`, `X_test`, `y_train`, `y_test` and `model_loaded` instead of `mdoel_loaded`.

4. Logistic Regression Concept – Interview Ready

Logistic Regression is a classification algorithm, not a regression algorithm in the usual ML sense. It models class probability using the logistic/sigmoid function and then applies a decision threshold to obtain a class label. Scikit-learn describes it as a linear model for classification. ■cite■turn0search0■turn0search1■

For binary classification, the sigmoid converts a linear score into a probability between 0 and 1. In multiclass classification such as Iris, Logistic Regression can handle multiple classes; scikit-learn uses multiclass strategies depending on the configuration. ■cite■turn0search1■

The notebook uses three Iris classes: `setosa`, `versicolor`, and `virginica`. The evaluation uses precision, recall, F1-score, and support through `classification_report`. ■cite■turn0search2■

5. Interview Questions and Short Answers

Q1. Is Logistic Regression used for regression?

No. Despite the name, it is primarily used for classification.

Q2. What is X in this project?

The four flower measurements: `sepal_length`, `sepal_width`, `petal_length`, and `petal_width`.

Q3. What is Y?

The species column, containing `setosa`, `versicolor`, and `virginica`.

Q4. Why split training and testing data?

Training data is used to learn the model; test data estimates performance on unseen observations.

Q5. Why use `random_state=21`?

To make the split reproducible so the same observations are selected each time.

Q6. Why use an imputer?

To provide a reusable strategy for missing numeric values. Median is less sensitive to extreme values than mean.

Q7. What does `model.score()` return here?

For Logistic Regression classification, it returns accuracy on the supplied data.

Q8. What is precision?

Among observations predicted as a class, precision is the fraction that actually belongs to that class.

Q9. What is recall?

Among actual observations of a class, recall is the fraction correctly identified.

Q10. What is F1-score?

The harmonic mean of precision and recall.

Q11. Why use macro F1?

It gives each class equal weight, which is useful when you want every class to matter equally.

Q12. Why save the pipeline?

So new data receives the same preprocessing learned during training before prediction.

6. Final ML Developer Summary

This notebook demonstrates a complete beginner-to-intermediate classification workflow on the Iris dataset. The final test accuracy is 93.33% and macro F1 is about 92.80%. The strongest production-oriented part is that the preprocessing pipeline and trained model are saved and later reused for new observations. The main technical cleanup is to make the target one-dimensional and to combine preprocessing plus model into one deployable pipeline.

Reference note: general Logistic Regression behavior and metric definitions in this guide are aligned with current scikit-learn documentation. Notebook-specific outputs and observations above come from the uploaded notebook itself.